

Algoritmy a grafy 1 (BI-AG1), Přednáška č. 7

Úvod do randomizace a hešování

Dušan Knop, Michal Opler, Tomáš Valla

Fakulta informačních technologií
České vysoké učení technické v Praze
<https://courses.fit.cvut.cz/BI-AG1/>



ZS 2025/2026, 3. listopadu 2025

(Verze dokumentu: 14. 12. 2025 19:21)

Slovníky

- Nejjednodušší datovou strukturou pro práci s **dynamickými množinami** je **slovník** (tabulka, mapa, asociativní pole).
- **Prvky** slovníku jsou stejně jako v předchozích přednáškách dvojice (**klíč, hodnota**). Klíč prvku x značíme zase $k(x)$.
- **Slovník** je datová struktura, která umožňuje reprezentovat dynamickou podmnožinu prvků s klíči $K \subseteq \mathcal{U}$, kde $|K| \ll |\mathcal{U}|$, a efektivně podporovat operace
 - ▶ **Find**(k): zjistí, zda prvek $k \in K$ (příp. vrať hodnotu prvku).
 - ▶ **Insert**(x): pokud $k(x) \notin K$, vlož x do slovníku.
 - ▶ **Delete**(x): pokud $k(x) \in K$, vymaž x ze slovníku.
- Prvky se tedy vyhledávají a ukládají podle svých klíčů.
- Klíče jsou **unikátní**, t.j., v daném okamžiku se vyskytuje nejvýše jeden prvek s daným klíčem.
- Množinu všech možných klíčů, které se mohou ve slovníku vyskytnout, nazveme **univerzum** $\mathcal{U}$.

Implementace slovníků

- Pro menší rozsahy univerza $\mathcal{U}$ lze použít například **bitové pole**.
- Potom časová složitost operací bude zjevně $O(1)$.
- Nicméně paměťové nároky budou $\Theta(|\mathcal{U}|)$.
- V praxi ale bývá velikost univerza $|\mathcal{U}|$ obrovské číslo.
- Podobně **tabulka s přímým adresováním** podle klíčů.
- Klasické slovníky (překladové, výkladové) používají (abecedně) **seřazená pole**: hledání je logaritmické, vkládání či mazání je lineární.
- V této přednášce se budeme věnovat **hešovacím tabulkám** (*hash tables*, alternativně se jim říká **rozptylovací tabulky**).
- Cílem hešování je skloubit nízké paměťové nároky operací, tzn. $O(|K|)$, a přitom zachovat konstantní složitost operací, i když pouze v průměrném případě.

Hešovací tabulky

- Pro nějaké univerzum klíčů $\mathcal{U}$, zvolme konečné pole **příhrádek** $\mathcal{P} = \{0, \dots, m-1\}$ (**hešovací tabulku**) a **hešovací funkci** $h: \mathcal{U} \rightarrow \mathcal{P}$, která každému klíči univerza přidělí jednu příhrádku.
- Chceme-li uložit množinu prvků s klíči $K \subseteq \mathcal{U}$, rozmístíme její prvky do příhrádek: prvek s klíčem $k \in K$ umístíme do příhrádky $h(k)$.
- Budeme-li hledat nějaký prvek s klíčem $k \in \mathcal{U}$, víme, že nemůže být jinde než v příhrádce $h(k)$.
- Díky poměru m a $|\mathcal{U}|$ ($m \ll |\mathcal{U}|$) se bude stávat, že několik prvků padne do stejné příhrádky.
- Tomu se říká **kolize**.
- Cílem je volit m a h tak, aby se počet kolizí minimalizoval.
- To, jak kolize řešit, vysvětlíme později.

Příklad hešovací tabulky

- Uvažujme univerzum všech celých čísel, hešovací tabulku s $m = 10$ přihrádkami a hešovací funkci $h(k) = k \bmod 10$.
- Celočíselné klíče budeme tedy rozdělovat do přihrádek podle poslední číslice.
- Předpokládejme, že jsme do tabulky vložili letopočty 1212, 935, 1948, 1918, 1968, 1989.

0	1	2	3	4	5	6	7	8	9
		1212			935			1948 1918 1968	1989

- Hledáme-li rok 2015, víme, že se musí nacházet v přihrádce 5. Tam je ovšem pouze 935, takže hned odpovíme zamítavě.
- Hledání roku 2017 je ještě rychlejší: přihrádka 7 je prázdná.
- Hledáme-li rok 1618, musíme ho porovnat s 3 prvky.

Ideální a dobře fungující hešovací funkce

- Složitost operací **Find**(k) a **Insert**(x) zjevně závisí na složitosti výpočtu hešovací funkce a na tom, jak rozmísťuje ukládané prvky do přihrádek.
- **Ideální hešovací funkce** $h: \mathcal{U} \rightarrow \mathcal{P}$ vypočte číslo přihrádky v **konstantním** čase a zadanou n -prvkovou množinu vstupních klíčů K rozprostře mezi m přihrádek $\mathcal{P}$ **dokonale rovnoměrně**, tzn. všechny přihrádky budou mít nejvýše $\lceil n/m \rceil$ prvků.
- Ideální hešovací funkce je obvykle nemožné sestrojít, proto se používají funkce, které se chovají „prakticky náhodně“.
- Ukážeme několik příkladů hešovacích funkcí, pro které bylo **experimentálně** ověřeno, že dobře hešují nejrůznější typy dat.

Příklady dobře fungujících hešovacích funkcí

- **Lineární kongruence:** $k \mapsto ak \bmod m$.
 - ▶ Zde m je typicky prvočíslo a a je nějaká dostatečně velká konstanta **nesoudělná** s m .
 - ▶ Často se a nastavuje blízko celé části $0.618m$.
- **Vyšší bity součinu:** $k \mapsto \lfloor (ak \bmod 2^w) / 2^{w-\ell} \rfloor$.
 - ▶ Pokud hešujeme w -bitové klíče do $m = 2^\ell$ přihrádek, vybereme w -bitovou lichou konstantu a (nesoudělnou s 2^w).
 - ▶ Pak pro každé k spočítáme ak , **ořízneme** ho na w bitů a z nich vezmeme **nejvyšších** ℓ .
 - ▶ Vzhledem k tomu, že přetečení ve většině programovacích jazyků automaticky ořezává výsledek, stačí k výpočtu hešovací funkce jedno násobení a bitový posun.

Dobře fungující hešovací funkce pro posloupnosti

- **Skalární součin:** $k_0, \dots, k_{d-1} \mapsto \left(\sum_{i=0}^{d-1} a_i k_i \right) \bmod m$.
 - ▶ Posloupnost klíčů zahešujeme tak, že zahešujeme každý klíč zvlášť a výsledky sečteme.
 - ▶ Pokud i -tý klíč k_i zahešujeme lineární kongruencí $k_i \mapsto a_i k_i \bmod m$, je heš celé posloupnosti její **skalární součin s vektorem konstant** $(a_0, a_1, \dots, a_{d-1})$ modulo m .
 - ▶ Podobně lze heš počítat v dvojkové soustavě pomocí operace XOR místo součtu.
- **Polynom:** $k_0, \dots, k_{d-1} \mapsto \left(\sum_{i=0}^{d-1} a^i k_i \right) \bmod m$.
 - ▶ Zvolíme jednu konstantu a a počítáme skalární součin zadané posloupnosti s **vektorem mocnin** $(a^0, a^1, \dots, a^{d-1})$ a opět modulo m .

Ideální hešování

- Přestože jsou vlastnosti hešovacích funkcí často postaveny na experimentálních výsledcích, pro některé případy hešování lze provést i exaktní analýzu.
- Požadované vlastnosti hešovací funkce:
 - 1) Výpočet $h(k)$ je rychlý ($O(1)$) a funkce h neukládá žádná data do paměti. Speciálně je tedy její výpočet nezávislý na historii předchozích volání.
 - 2) Hešovací funkce h **rozděluje univerzum $\mathcal{U}$ do přihrádek rovnoměrně**, například takto:

$$\forall i \neq j \in \{0, \dots, m-1\} : |h^{-1}(i)| = |h^{-1}(j)| \pm 1.$$

Časová složitost ideálního hešování

Fakt (neformální)

Uvažujme hešovací tabulku velikosti m s n prvky a s hešovací funkcí h a necht' jsou splněny předpoklady 1) a 2).

Necht' navíc platí, že vstupní data jsou z univerza $\mathcal{U}$ vybírána nezávisle rovnoměrně náhodně.

Potom

- 1 průměr počtu prvků v přihrádce je n/m ,
- 2 počet prvků v přihrádce je $O(n/m)$ pro skoro všechny přihrádky a
- 3 průměrný počet operací vykonaných při hledání, vkládání i mazání je $O(n/m)$.

Nafukovací hešovací tabulka a přehešování

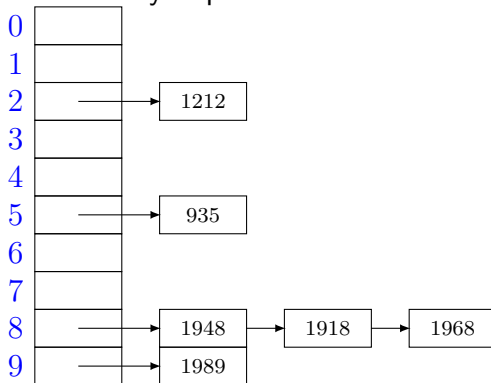
- Předpokládejme, že počet zahešovaných prvků n předem neznáme a nedokážeme jej rozumně odhadnout.
- Jak máme zvolit počet přihrádek m hešovací tabulky, pokud budeme chtít zajistit, aby **faktor naplnění** hešovací tabulky $\alpha = n/m$ nepřekročil zadanou konstantu Z ?
- Pomůže nám technika **amortizovaného nafukovacího pole**.
 - ▶ Na počátku založíme prázdnou hešovací tabulku s nějakým počátečním počtem přihrádek $m = m_0$.
 - ▶ Po vložení prvku zkontrolujeme $\alpha = n/m$.
 - ▶ Pokud $\alpha > Z$, je hešovací tabulka přeplněná. Pak zdvojnásobíme m , zvolíme novou hešovací funkci a všechny prvky přehešujeme: pro každý prvek ze staré tabulky spočteme novou hešovací funkci a vložíme do nové tabulky.

Řešení kolizí

- Jelikož $m \ll |\mathcal{U}|$, žádná hešovací funkce se nemůže vyhnout kolizím, kdy je několik prvků zahešováno do téže přihrádky.
- Existují dvě principiálně odlišné metody řešení kolizí:
 - 1 **Řetězení** (*Chaining*) = **Otevřené hešování** (*Open hashing*).
Otevřené hešování proto, že ukládané prvky nejsou ukládány do hešovací tabulky, ale do „otevřeného prostoru“ za ní, do **řetízků**, odkazovaných z tabulky.
 - 2 **Otevřené adresování** (*Open addressing*) = **Uzavřené hešování** (*Closed hashing*). Uzavřené hešování proto, že prvky jsou ukládány do „uzavřeného prostoru tabulky“. Otevřené adresování proto, že číslo (index) přihrádky tabulky, kam je nakonec hešovaný prvek uložen, je dopředu otevřený, neboť není dán pouze hešovací funkcí, ale i momentální obsazeností tabulky.

Řešení kolizí řetězením

- Hešovací tabulka je **pole** m přihrádek, které jsou buď **prázdné** nebo obsahují **ukazatel** na **řetízky** (= spojové seznamy) uložených prvků.
- Jedná se o přímočarou implementaci abstraktní struktury hešovací tabulky.
- Naše tabulka bude tedy implementovaná takto:



Vlastnosti hešování s řetízky

Časová složitost:

- Hledání, vkládání i mazání sestává z výpočtu hešovací funkce a projití řetízku v příslušné přihrádce.
- V případě ideální hešovací funkce a pokud jsou vstupní data z univerza $\mathcal{U}$ vybírána nezávisle rovnoměrně náhodně, budou mít (skoro) všechny řetízky délku nejvýše $\lceil n/m \rceil$ prvků.
- Zvolíme-li navíc počet přihrádek $m = \Theta(n)$, vyjde konstantní délka řetízku, a tím pádem i časová složitost operací.

Otevřená adresace: Řešení kolizí

- **Hešovací tabulka s otevřenou adresací** je **pole** s m přihrádkami $A[0], \dots, A[m-1]$, jenž tentokrát se do každé přihrádky vejde **pouze jeden prvek**.
- Při pokusu o uložení do obsazené přihrádky budeme postupně zkoušet náhradní přihrádky, dokud nenajdeme prázdnou.
- Hešovací funkce tedy každému klíči $k \in \mathcal{U}$ přiřadí jeho **vyhledávací posloupnost** $h(k, 0), h(k, 1), \dots, h(k, m-1)$, která je ideálně **permutací** posloupnosti $\{0, 1, \dots, m-1\}$.
- Ta určuje pořadí přihrádek, do kterých se budeme postupně snažit vkládat klíč k .
- V ideálním případě vyhledávací posloupnost obsahuje všechna čísla přihrádek v dokonale náhodném pořadí (všechny permutace přihrádek jsou stejně pravděpodobné).

Otevřená adresace: Hledání

OpenFind(k)

(1) Pro $i = 0, \dots, m - 1$:

(2) $j := h(k, i)$

(3) Pokud $k(A[j]) = k$: ohlaš nalezení $A[j]$ a skonči

(4) Pokud je $A[j]$ prázdná: ohlaš neúspěch a skonči

(5) Ohlaš neúspěch

Otevřená adresace: Mazání

- Mazání je problematické. Pokud bychom přihrádku smazaného prvku označili jako prázdnou, můžeme způsobit, že **existující** prvek s nějakým jiným klíčem k' **nebude nalezen**, protože vyhledávací posloupnost, která byla použita při vkládání k' , bude tímto zásahem **přerušena**.

Příklad

Nechť $m = 5$, $k' = 1$ a předpokládejme vyhledávací posloupnost $h(k', 0) = 0$, $h(k', 1) = 3$, $h(k', 2) = 1$, ...

Hešovací tabulka:

0	1	2	3	4
7	1	9	4	

OpenDeleteNaive(4):

0	1	2	3	4
7	1	9		

OpenFind(k'): neúspěch
⇒ **chybný výsledek**

0	1	2	3	4
7	1	9		



Otevřená adresace: Mazání

- Proto budeme mazané prvky **pouze označovat** za smazané.
- Standardní implementace označení smazaného prvku je pomocí tzv. **náhrobku** $\dagger$ (*tombstone*).
- Dodefinujeme $k(\dagger)$ jako libovolnou hodnotu mimo $\mathcal{U}$.

Pokračování příkladu

Předpokládejme stále $m = 5$, $k' = 1$, vyhledávací posloupnost $h(k', 0) = 0$, $h(k', 1) = 3$, $h(k', 2) = 1$,

Hešovací tabulka:

0	1	2	3	4
7	1	9	4	

OpenDelete(4):

0	1	2	3	4
7	1	9	$\dagger$	

OpenFind(k'):
 $\Rightarrow$ úspěch

0	1	2	3	4
7	1	9	$\dagger$	



Otevřená adresace: Vkládání

- Místo po smazaných prvcích můžeme využívat při vkládání.
- Operace **OpenInsert** je pak následující:

OpenInsert(x)

- (1) Pokud **OpenFind**($k(x)$) nalezne $k(x)$: skonči
- (2) Pro $i = 0, \dots, m - 1$:
- (3) $j := h(k(x), i)$
- (4) Pokud je $A[j]$ prázdná nebo $A[j] = \dagger$:
- (5) $A[j] := x$ a skonči
- (6) Ohlaš zaplnění hešovací tabulky

- Pokud se vyhledávací posloupnosti nově vkládaných prvků liší od vyhledávacích posloupností mazaných prvků označovaných náhrobky, počet náhrobků může snadno přesáhnout zadanou mez (třeba $m/4$). Pak se obvykle struktura tabulky přebuduje (**přehešuje**).

Otevřená adresace: Mazání

- Prvek nahradíme $\dagger$.
- Operace **OpenDelete** je pak následující:

OpenDelete(x)

- (1) Pro $i = 0, \dots, m - 1$:
- (2) $j := h(k(x), i)$
- (3) Pokud je $A[j] = x$:
- (4) $A[j] = \dagger$
- (5) skonči
- (6) Pokud je $A[j]$ prázdná:
- (7) skonči

Volba vyhledávací posloupnosti

- Jak volit vyhledávací posloupnosti?
- Ukážeme základní možnosti, které se často používají v praxi.

Lineární přidávání

Lineární přidávání (*Linear Probing*): Vyhledávací posloupnost je dána funkcí $h(k, i) = (f(k) + i) \bmod m$, kde $f(k)$ je „obyčejná“ hešovací funkce a i je počet neúspěšných pokusů v aktuální operaci.

- Zkoušíme tedy jednoduše za sebou jdoucí přihrádky.
- Výhodou je sekvenční přístup do paměti, který je na dnešních počítačích se skrytými paměťmi (cache memory) rychlejší.
- Nevýhodou je to, že jakmile se vytvoří souvislé bloky obsazených přihrádek, další vkládání se do nich často strefí a bloky dále porostou.
- Bez důkazu uvádíme, že pro neúspěšné hledání platí odhad průměrného počtu navštívených přihrádek $1/(1 - \alpha)^2$. a to pouze, je-li f dokonale náhodná.
- Není-li, chování struktury obvykle degraduje.
- Lze zobecnit na **lineární přidávání s krokem**, kde $h(k, i) = (f(k) + c \cdot i) \bmod m$ a $c > 1$ je konstanta nesoudělná s m .

Dvojité hešování

Dvojité hešování (*Double Hashing*): Vyhledávací posloupnost je dána funkcí $h(k, i) = (f(k) + i \cdot g(k)) \bmod m$, kde $f: \mathcal{U} \rightarrow \{0, \dots, m-1\}$ a $g: \mathcal{U} \rightarrow \{1, \dots, m-1\}$ jsou dvě různé hešovací funkce, m je prvočíslo a i je počet neúspěšných pokusů v aktuální operaci

- Protože m je prvočíslo, je s ním $g(k)$ vždy nesoudělné a vyhledávací posloupnost navštíví každou přihrádku právě jednou.
- Je známo, že pro dokonale náhodné funkce f a g se dvojité hešování chová stejně dobře, jako při použití plně náhodných vyhledávacích posloupností.
- Toto tvrzení též ponecháváme bez důkazu.

Efektivita hešování s otevřenou adresací

Věta (O efektivitě otevřené adresace)

Pokud jsou vyhledávací posloupnosti náhodné permutace, pak neúspěšné hledání nahlédne v průměru do **nejvýše** $1/(1 - \alpha)$ přihrádek, kde $\alpha = n/m$, $n < m$, je faktor naplnění.

- Důkaz využívá základní pojmy z teorie pravděpodobnosti.
- Ty si pro účely důkazu musíme zavést.
- To nám dovolí také využívat náhodu ve výpočtech.
- Základní vlastností a výhodou **deterministických** algoritmů je to, že jejich opakované spuštění nad stejnými daty proběhne vždy stejně se stejnými výsledky.
- V řešení řady problémů je ale užitečné zavést do výpočtu prvek náhody, provést jejich tzv. **randomizaci**.
- Dobrý příklad jsou datové struktury, u kterých se náhodnost používá např. jako obrana před zlomyslným útočníkem.

Náhoda v algoritmech

- Příklad:
 - ▶ Na k serverů chodí výpočetně různě náročné požadavky.
 - ▶ Když by systém vyvažování výpočetní zátěže (load balancer) rozesílal tyto požadavky na servery ve stále stejném pevném pořadí $1, 2, \dots, k$, mohl by útočník fakticky vyřadit z provozu jeden ze serverů tím, že zajistí, aby každý k -tý požadavek byl výpočetně velmi náročný.
 - ▶ Triviální obrana proti takovému útoku je zavést do výpočtu prvek náhody: servery budou vybírány **náhodně**.
- Pro analýzu algoritmů využívajících náhodné hodnoty proměnných je třeba použít nástroje **teorie pravděpodobnosti**, jejíž základy pro potřeby této přednášky nyní vysvětlíme.
- (Všechny tyto základní pojmy budou podrobněji probrány v předmětu BI-PST Pravděpodobnost a statistika v 5.semestru).

Zavedení pojmu pravděpodobnost

Definice

Diskrétní pravděpodobnostní prostor je dvojice $(\Omega, \mathbf{P})$, kde

- Ω je konečná nebo spočetně nekonečná množina **elementárních jevů**.
- $\mathbf{P} : \Omega \rightarrow [0, 1]$ je funkce, která elementárním jevům přiřazuje jejich **pravděpodobnosti**, taková, že součet pravděpodobností všech elementárních jevů je roven 1, čili $\sum_{\omega \in \Omega} \mathbf{P}(\omega) = 1$.

Jev je nějaká množina $A \subseteq \Omega$ elementárních jevů.

Pravděpodobnost jevu A je: $\mathbf{P}(A) = \sum_{\omega \in A} \mathbf{P}(\omega)$.

- Pravděpodobnosti můžeme také připisovat výrokům: $\mathbf{P}[\varphi(\omega)]$ je pravděpodobnost jevu daného množinou všech elementárních jevů ω , pro které platí výrok $\varphi(\omega)$.
- BI-DML/PST: Uvažujeme, že množina $\mathcal{F} = \mathcal{P}(\Omega)$.

Obecněji viz BI-PST, 1. přednáška.

Příklady pravděpodobnostního prostoru

Pravděpodobnostní prostor šestistěnné kostky

- $\Omega_6 = \left\{ \begin{array}{|c|c|} \hline \cdot & \\ \hline \end{array}, \begin{array}{|c|c|} \hline \cdot & \cdot \\ \hline \end{array}, \begin{array}{|c|c|} \hline \cdot & \cdot & \cdot \\ \hline \end{array}, \begin{array}{|c|c|} \hline \cdot & \cdot & \cdot & \cdot \\ \hline \end{array}, \begin{array}{|c|c|} \hline \cdot & \cdot & \cdot & \cdot & \cdot \\ \hline \end{array}, \begin{array}{|c|c|} \hline \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \hline \end{array} \right\}.$
- $\mathbf{P}(\omega) = 1/6$ pro každé $\omega \in \left\{ \begin{array}{|c|c|} \hline \cdot & \\ \hline \end{array}, \begin{array}{|c|c|} \hline \cdot & \cdot \\ \hline \end{array}, \begin{array}{|c|c|} \hline \cdot & \cdot & \cdot \\ \hline \end{array}, \begin{array}{|c|c|} \hline \cdot & \cdot & \cdot & \cdot \\ \hline \end{array}, \begin{array}{|c|c|} \hline \cdot & \cdot & \cdot & \cdot & \cdot \\ \hline \end{array}, \begin{array}{|c|c|} \hline \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \hline \end{array} \right\}.$
- Jev J_L definovaný jako „padlo liché číslo“:
 $J_L = \left\{ \begin{array}{|c|c|} \hline \cdot & \\ \hline \end{array}, \begin{array}{|c|c|} \hline \cdot & \cdot & \cdot \\ \hline \end{array}, \begin{array}{|c|c|} \hline \cdot & \cdot & \cdot & \cdot \\ \hline \end{array} \right\} \subseteq \Omega$ a $\mathbf{P}(J_L) = 3 \cdot \frac{1}{6} = \frac{1}{2}.$

Pravděpodobnostní prostor čtyřstěnné kostky

- $\Omega_4 = \left\{ \begin{array}{|c|c|} \hline \cdot & \\ \hline \end{array}, \begin{array}{|c|c|} \hline \cdot & \cdot \\ \hline \end{array}, \begin{array}{|c|c|} \hline \cdot & \cdot & \cdot \\ \hline \end{array}, \begin{array}{|c|c|} \hline \cdot & \cdot & \cdot & \cdot \\ \hline \end{array} \right\}.$
- $\mathbf{P}(\omega) = 1/4$ pro každé $\omega \in \left\{ \begin{array}{|c|c|} \hline \cdot & \\ \hline \end{array}, \begin{array}{|c|c|} \hline \cdot & \cdot \\ \hline \end{array}, \begin{array}{|c|c|} \hline \cdot & \cdot & \cdot \\ \hline \end{array}, \begin{array}{|c|c|} \hline \cdot & \cdot & \cdot & \cdot \\ \hline \end{array} \right\}.$
- Jev J_S definovaný jako „padlo sudé číslo“:
 $J_S = \left\{ \begin{array}{|c|c|} \hline \cdot & \cdot & \cdot \\ \hline \end{array}, \begin{array}{|c|c|} \hline \cdot & \cdot & \cdot & \cdot \\ \hline \end{array} \right\} \subseteq \Omega_4$ a $\mathbf{P}(J_S) = 2 \cdot \frac{1}{4} = \frac{1}{2}.$

Základní vlastnosti pravděpodobnosti

Pozorování

Pro každé dva jevy A, B platí

$$\mathbf{P}(A \cup B) = \mathbf{P}(A) + \mathbf{P}(B) - \mathbf{P}(A \cap B).$$

Definice

Dva jevy A, B nazveme **nezávislé**, pokud $\mathbf{P}(A \cap B) = \mathbf{P}(A) \cdot \mathbf{P}(B)$.

Definice

Jev $\overline{A} = \Omega \setminus A$ se nazývá jev **opačný** k jevu A .

Pozorování

$$\mathbf{P}(\overline{A}) = 1 - \mathbf{P}(A).$$

Obecněji opět viz BI-PST, 1. přednáška.

Randomizovaný výpočetní model

- Abychom mohli náhodný výběr používat v rámci algoritmů, rozšíříme definici výpočetního modelu RAM doplněním nové instrukce $X := \text{random}(Y, Z)$, která vygeneruje náhodné celé číslo z intervalu $\langle Y, Z \rangle$ a uloží ho do X .
- Všechny hodnoty z tohoto intervalu jsou stejně pravděpodobné a volba je nezávislá na předchozích voláních, t.j., instrukce $\text{random}()$ nemá paměť.

Jaké vlastnosti může mít (rozhodovací) algoritmus využívající náhodné bity během svého výpočtu?

- 1 Pracuje vždy správně, ale celková doba výpočtu je závislá na náhodných bitech.
- 2 Celková doba výpočtu je nezávislá na náhodných bitech, ale algoritmus může občas odpovědět **nevím** (nebo se může občas „mýlit“).

Kontrola násobení matic

Mějme (racionální) matice A, B tvaru $n \times n$.

Naleznout matici $C = A \cdot B$:

- lze „školním algoritmem“ pomocí $O(n^3)$ aritmetických operací,
- lze Strassenovým algoritmem (1969) pomocí $O(n^{\log 7}) \approx O(n^{2,807})$ aritmetických operací,
- ...
- lze¹ pomocí $O(n^\omega)$ aritmetických operací, kde $\omega < 2,371339$.

Pokročilé algoritmy se vyplatí jen pro opravdu velké matice.

Jak si ale můžeme být jistí, že je výsledek správně?

- Například výsledek „uzavřené“ knihovní funkce?

¹Alman, Josh; Duan, Ran; Williams, Virginia Vassilevska; Xu, Yinzhan; Xu, Zixuan; Zhou, Renfai (2025), *More Asymmetry Yields Faster Matrix Multiplication*, 36th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2025), arXiv:2404.16349

Kontrola násobení matic

Mějme (racionální) matice A, B, C tvaru $n \times n$.

Chceme ověřit, že platí $C = A \cdot B$:

VerifyMM(A, B, C)

(1) Zvol náhodný vektor $x \in \{0, 1\}^n$

(2) $y = A \cdot (B \cdot x)$

(3) $z = Cx$

(4) Pokud $z == y$:

(5) return „ $C = A \cdot B$ “

(6) return „ $C \neq A \cdot B$ “

Věta

Algoritmus **VerifyMM** vykoná $O(n^2)$ aritmetických operací. Pokud

- ① $C = A \cdot B$, algoritmus jistě odpoví „ $C = A \cdot B$ “,
- ② $C \neq A \cdot B$, pak algoritmus odpoví „ $C \neq A \cdot B$ “ s pravděpodobností alespoň $\frac{1}{2}$.

Kontrola násobení matic

Důkaz.

Pokud $C = A \cdot B$, pak pro všechna $x \in \{0, 1\}^n$ platí $Cx = A \cdot Bx$. Předpokládejme, že $C \neq A \cdot B$. Položme $D = (C - AB)$.

Chceme ukázat, že pro libovolnou nenulovou matici D a náhodný vektor $x \in \{0, 1\}^n$ platí $\mathbf{P}[Dx \neq 0] \geq \frac{1}{2}$.

Bud' $d_{i,j} \neq 0$ a označme $y = D \cdot x$. Tvrdíme, že $\mathbf{P}[y_i \neq 0] \geq \frac{1}{2}$. Máme

$$y_i = d_{i,1}x_1 + d_{i,2}x_2 + \cdots + d_{i,n}x_n = d_{i,j}x_j + S.$$

Volíme x po složkách. Předpokládejme, že x_j volíme jako poslední. Před posledním hodem je hodnota S zafixovaná a nezávisí na x_j . Nyní $y_i = S$ nebo $y_i = S + d_{i,j}$. Alespoň jedna z těchto hodnot je nenulová (a každou vrátíme s pravděpodobností $1/2$). □

Kontrola násobení matic

Věta

Algoritmus **VerifyMM** vykoná $O(n^2)$ aritmetických operací. Pokud

- 1 $C = A \cdot B$, algoritmus jistě odpoví „ $C = A \cdot B$ “,
- 2 $C \neq A \cdot B$, pak algoritmus odpoví „ $C \neq A \cdot B$ “ s pravděpodobností alespoň $\frac{1}{2}$.

Pravděpodobnost $1/2$ ale není mnoho!

Pro zvýšení pravděpodobnosti správné odpovědi lze spustit algoritmus vícekrát:

Pokud **VerifyMM** spuštěný $50\times$ odpoví „ $C = A \cdot B$ “, pak

$$\mathbf{P}[C \neq A \cdot B] \leq (1/2)^{50} < 10^{-15}.$$

To je pro praktické výpočty postačující a složitost algoritmu zůstává $O(n^2)$.

Efektivita hešování s otevřenou adresací

Věta (O efektivitě otevřené adresace)

Pokud jsou vyhledávací posloupnosti náhodné permutace, pak neúspěšné hledání nahlédne ve střední hodnotě do **nejvýše** $1/(1 - \alpha)$ přihrádek, kde $\alpha = n/m$, $n < m$, je faktor naplnění.

Důkaz.

- Nechť k je hledaný klíč a $h_1, h_2, \dots, h_m$ jeho vyhledávací posloupnost.
- Označme p_i pravděpodobnost toho, že během hledání projdeme a zkusíme **alespoň** i přihrádek. Tedy pravděpodobnost toho, že přihrádky $h_1, \dots, h_{i-1}$ jsou obsazené.
- ...

Pokračování důkazu.

- Příhradku h_1 projdeme vždy, proto $p_1 = 1$.
- Pravděpodobnost p_2 je pravděpodobnost toho, že příhrádka h_1 (náhodně vybraná z m možností) je obsazena libovolným klíčem z n možných.
- Proto $p_2 = n/m = \alpha$.
- Obecně p_{i+1} je pravděpodobnost toho, že i příhrádka h_i je obsazená, stejně jako $i - 1$ předchozích.
- Čili náhodně vybraná příhrádka ze zbývajících $m - (i - 1)$ možných příhrádek je obsazena libovolným klíčem ze zbývajících $n - (i - 1)$ možných klíčů.
- Proto $p_{i+1} = p_i \cdot (n - i + 1)/(m - i + 1) \leq p_i \cdot n/m = p_i \cdot \alpha$.
- Indukcí dostaneme $p_{i+1} \leq \alpha^i$.
- ...

Pokračování důkazu.

- Nechť S = střední hodnota počtu navštívených přihrádek.
- Pak $S = \sum_{i=1}^m i \cdot q_i$, kde q_i je pravděpodobnost, že jsme navštívili **právě** i přihrádek. Tedy, že přihrádky $h_1, \dots, h_{i-1}$ jsou obsazené a přihrádka h_i je prázdná.
- Protože $\mathbf{P}[h_1, \dots, h_{i-1} \text{ jsou obsazené}] = \mathbf{P}[h_1, \dots, h_{i-1} \text{ jsou obsazené a } h_i \text{ je prázdná}] + \mathbf{P}[h_1, \dots, h_{i-1} \text{ jsou obsazené a } h_i \text{ je obsazená}]$, platí $p_i = q_i + p_{i+1}$ čili $q_i = p_i - p_{i+1}$. Proto

$$\begin{aligned} S &= \sum_{i=1}^m i \cdot (p_i - p_{i+1}) = \sum_{i=1}^m i \cdot p_i - \sum_{i=1}^m i \cdot p_{i+1} = \\ &= \sum_{i=1}^m i \cdot p_i - \sum_{i=2}^m (i-1) \cdot p_i = \sum_{i=1}^m p_i \cdot (i - (i-1)) \\ &= \sum_{i=1}^m p_i. \end{aligned}$$

• ...

Dokončení důkazu.

- Tedy

$$S = \sum_{i=1}^m p_i \leq \sum_{i=1}^m \alpha^{i-1} < \sum_{i=0}^{\infty} \alpha^i = 1/(1 - \alpha).$$

neboť $\alpha < 1$ a jedná se tedy o konvergentní geometrickou řadu.



- Jelikož $n < m$, platí $\forall i \geq n + 2; p_i = 0$ a $\forall i \geq n + 2; q_i = 0$.
- Tudíž $q_{n+1} = p_{n+1}$.

Náhodná veličina

Pokud pro vstup **VerifyMM** platí $C \neq A \cdot B$, kolikrát musíme v průměru algoritmus spustit než dostaneme korektní odpověď?

Toto budeme využívat i pro analýzu průměrné doby běhu randomizovaných algoritmů.

Definice

Nechť $(\Omega, \mathbf{P})$ je diskrétní pravděpodobnostní prostor. Pak **diskrétní náhodná veličina** je funkce $X: \Omega \rightarrow \mathbb{R}$.

- Diskrétní náhodná veličina tedy přiřazuje každému elementárnímu jevu $\omega_i \in \Omega$ nějakou číselnou hodnotu $x_i = X(\omega_i)$ a může nabývat pouze konečný nebo spočetně nekonečný počet hodnot $\{x_1, x_2, \dots\}$.
- Náhodná veličina se také někdy nazývá **náhodná proměnná**.

Obecněji viz BI-PST, 3. přednáška.

Příklad náhodné veličiny

Nad pravděpodobnostním prostorem šestistěnné kostky lze celkem přirozeně definovat náhodnou veličinu K , která každému hodu kostkou přiřadí počet „teček“, které padly. Čili

$K : \left\{ \begin{array}{|c|} \hline \bullet \\ \hline \end{array}, \begin{array}{|c|} \hline \bullet \bullet \\ \hline \end{array}, \begin{array}{|c|} \hline \bullet \bullet \bullet \\ \hline \end{array}, \begin{array}{|c|} \hline \bullet \bullet \bullet \bullet \\ \hline \end{array}, \begin{array}{|c|} \hline \bullet \bullet \bullet \bullet \bullet \\ \hline \end{array}, \begin{array}{|c|} \hline \bullet \bullet \bullet \bullet \bullet \bullet \\ \hline \end{array} \right\} \rightarrow \mathbb{R}$, kde

- $K\left(\begin{array}{|c|} \hline \bullet \\ \hline \end{array}\right) = 1$,
- $K\left(\begin{array}{|c|} \hline \bullet \bullet \\ \hline \end{array}\right) = 2$,
- $K\left(\begin{array}{|c|} \hline \bullet \bullet \bullet \\ \hline \end{array}\right) = 3$,
- $K\left(\begin{array}{|c|} \hline \bullet \bullet \bullet \bullet \\ \hline \end{array}\right) = 4$,
- $K\left(\begin{array}{|c|} \hline \bullet \bullet \bullet \bullet \bullet \\ \hline \end{array}\right) = 5$,
- $K\left(\begin{array}{|c|} \hline \bullet \bullet \bullet \bullet \bullet \bullet \\ \hline \end{array}\right) = 6$.

Střední hodnota náhodné veličiny

Definice

Střední hodnota $\mathbf{E}[X]$ náhodné veličiny X je průměr všech hodnot veličiny X vážený pravděpodobnostmi příslušných elementárních jevů, tedy
$$\sum_{\omega \in \Omega} X(\omega) \cdot \mathbf{P}(\omega) = \sum_{x_i} x_i \cdot \mathbf{P}[X = x_i].$$

Tato řada musí konvergovat absolutně, jinak střední hodnotu nezavádíme.

Obecněji viz BI-PST, 4. přednáška.

Příklad:

Střední hodnota náhodné veličiny K z předchozího slajdu, čili střední hodnota počtu teček padlých na šestistěnné kostce, je
$$\mathbf{E}[K] = \sum_{\omega \in \Omega_6} K(\omega) \cdot P(\omega) = 1 \cdot \frac{1}{6} + 2 \cdot \frac{1}{6} + \dots + 6 \cdot \frac{1}{6} = \frac{21}{6} = 3,5.$$

Podobně, střední hodnota náhodné veličiny C definované jako počet teček padlých na čtyřstěnné kostce je

$$\mathbf{E}[C] = 1 \cdot \frac{1}{4} + 2 \cdot \frac{1}{4} + 3 \cdot \frac{1}{4} + 4 \cdot \frac{1}{4} = \frac{10}{4} = 2,5.$$

Linearity střední hodnoty

Věta (O linearitě střední hodnoty)

Nechť α a β jsou reálná čísla a necht' X a Y jsou náhodné veličiny. Potom $\mathbf{E}[\alpha X + \beta Y] = \alpha \mathbf{E}[X] + \beta \mathbf{E}[Y]$.

Důkaz pro případ stejného pravděpodobnostního prostoru.

Bud' $(\Omega, \mathbf{P})$ diskrétní pravděpodobnostní prostor.

$$\begin{aligned}\mathbf{E}[\alpha X + \beta Y] &= \sum_{\omega \in \Omega} (\alpha X + \beta Y)(\omega) \cdot \mathbf{P}(\omega) = \\ &= \sum_{\omega \in \Omega} (\alpha \cdot X(\omega) + \beta \cdot Y(\omega)) \cdot \mathbf{P}(\omega) = \\ &= \sum_{\omega \in \Omega} \alpha \cdot X(\omega) \cdot \mathbf{P}(\omega) + \sum_{\omega \in \Omega} \beta \cdot Y(\omega) \cdot \mathbf{P}(\omega) = \\ &= \alpha \cdot \sum_{\omega \in \Omega} X(\omega) \cdot \mathbf{P}(\omega) + \beta \cdot \sum_{\omega \in \Omega} Y(\omega) \cdot \mathbf{P}(\omega) = \\ &= \alpha \mathbf{E}[X] + \beta \mathbf{E}[Y]. \quad \square\end{aligned}$$

Příklad použití linearity střední hodnoty

- Uvažujme házení dvěma hracími kostkami současně: šestistěnnou kostkou s 1 až 6 tečkami a čtyřstěnnou kostkou s 1 až 4 tečkami.
- Tomu bude odpovídat pravděpodobnostní prostor $(\Omega, \mathbf{P})$, kde

$$\Omega = \{(x, y); x \in \{\square, \dots, \begin{smallmatrix} \blacksquare \\ \blacksquare \\ \blacksquare \end{smallmatrix}\}, y \in \{\square, \dots, \begin{smallmatrix} \blacksquare & \blacksquare \\ \blacksquare & \blacksquare \end{smallmatrix}\}\}$$

je množina uspořádaných dvojic počtu teček na jednotlivých kostkách a $\mathbf{P}(\omega) = \frac{1}{6} \cdot \frac{1}{4} = \frac{1}{24}$.

- Definujme opět náhodnou veličinu K jako počet teček, který padne na první kostce, a náhodnou veličinu C jako počet teček, který padne na druhé kostce.
- Na Slajdu 40 jsme spočítali $\mathbf{E}[K] = 1 \cdot 1/6 + \dots + 6 \cdot 1/6 = 3,5$ a $\mathbf{E}[C] = 2,5$.
- Pomocí linearity střední hodnoty můžeme snadno spočítat střední hodnotu **součtu počtu teček na obou kostkách**:
 $\mathbf{E}[K + C] = \mathbf{E}[K] + \mathbf{E}[C] = 3,5 + 2,5 = 6$.

Čekání na náhodný jev

V algoritmech často využíváme následující skutečnost: Čekáme-li na událost, která v každém pokusu nastává s pravděpodobností p , bude **střední počet pokusů, než událost nastane** roven $1/p$.

Fakt (Věta o opakování nezávislých pokusů)

Uvažujme sérii **nezávislých** pokusů, ve kterých sledujeme, zda nastal nějaký jev J . Pravděpodobnost, že jev J nastane, je v každém pokusu stále stejná a rovna p . Definujme náhodnou veličinu T jako **pořadí** pokusu, ve kterém jev J nastal **poprvé**. Pak $\mathbf{E}[T] = 1/p$.

Důkaz bude v BI-PST.

Pokud pro vstup **VerifyMM** platí $C \neq A \cdot B$, kolikrát musíme v průměru algoritmus spustit než dostaneme korektní odpověď?

$$\mathbf{E}[T] = 1/p = \frac{1}{\frac{1}{2}} = 2.$$

Co bychom po dnešku už měli znát

- Slovník.
- Hešování, hešovací funkce, hešovací tabulka, kolize.
- Příklady praktických hešovacích funkcí.
- Faktor naplnění, přehešování.
- Hešování s řetízky.
- Hešování s otevřenou adresací, vyhledávací posloupnost, náhrobek.
- Efektivita ideálních vyhledávacích posloupností.
- Lineární přidávání, dvojité hešování.
- Úvod do teorie pravděpodobnosti.
- Užití pravděpodobnosti ke kontrole násobení matic.
- Střední hodnota.